WEEK-10

OSSP - SKILL Content Delivery Concepts

Course Code: 25CS2104E

TASK 1

Objective / Skill:

To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs.

Implementation / Work:

```
/*
 * Week 10 - Task 1: Combined Redirection & Stream Merging (2>&1)
 * Objective: To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams,
 *            Verify Ordering, Handle Edge Cases, Test Integrated Outputs.
 */
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    printf("=== Week 10 Task 1: Combined Redirection & 2>&1 Merging ===\n\n");
    const char *combined_file = "combined_output.log";

    fflush(stdout);
    int saved_stdout = dup(STDOUT_FILENO);
    int saved_stderr = dup(STDERR_FILENO);

    int fd = open(combined_file, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) { perror("open failed"); return 1; }

    // Redirect stdout to file, then merge stderr into stdout (2>&1)
    dup2(fd, STDOUT_FILENO);
    dup2(STDOUT_FILENO, STDERR_FILENO);
    close(fd);

    printf("[STDOUT] Normal informational message processed.\n");
    fflush(stdout);   // flush in order so the log preserves write ordering
    fprintf(stderr, "[STDERR] Warning diagnostic output merged into same stream.\n");
    fflush(stderr);

    dup2(saved_stdout, STDOUT_FILENO);
    dup2(saved_stderr, STDERR_FILENO);
    close(saved_stdout);
    close(saved_stderr);

    printf("Streams restored. Verifying combined log content:\n");
    printf("----------------------------------------\n");
    FILE *f = fopen(combined_file, "r");
    if (f) {
        char line[128];
        while (fgets(line, sizeof(line), f)) printf("  %s", line);
        fclose(f);
    }
    printf("----------------------------------------\nStream merging verified!\n");
    return 0;
}
```

Output:

```
charith@DESKTOP-RBQQSS:~$ nano s10.1.c
charith@DESKTOP-RBQQSS:~$ gcc s10.1.c -o s10.1
charith@DESKTOP-RBQQSS:~$ ./s10.1
=== Week 10 Task 1: Combined Redirection & 2>&1 Merging ===

Streams restored. Verifying combined log content:
----------------------------------------
  [STDOUT] Normal informational message processed.
  [STDERR] Warning diagnostic output merged into same stream.
----------------------------------------
Stream merging verified!
charith@DESKTOP-RBQQSS:~$
```

TASK 2

Objective / Skill:

To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios.

Implementation / Work:

```
/*
 * Week 10 - Task 2: Execution Planning for Combined Pipes and Redirection
 * Objective: To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order,
 *            Validate Syntax, Generate Execution Plans, Test Scenarios.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char command[64];
    char in_file[32];
    char out_file[32];
    int has_pipe_out;
} StepPlan;

static char *trim(char *s) {
    while (*s == ' ' || *s == '\t') s++;
    char *e = s + strlen(s);
    while (e > s && (e[-1] == ' ' || e[-1] == '\t' || e[-1] == '\n')) *--e = '\0';
    return s;
}

/* Split one pipeline stage into command text, '<' file and '>' file */
static void parse_stage(char *stage, StepPlan *p) {
    memset(p, 0, sizeof(*p));
    char *in = strchr(stage, '<');
    char *out = strchr(stage, '>');
    if (in)  { *in = '\0';  strncpy(p->in_file,  trim(in + 1), sizeof(p->in_file) - 1); }
    if (out) { *out = '\0'; strncpy(p->out_file, trim(out + 1), sizeof(p->out_file) - 1); }
    /* a '<' that appears after '>' leaves its file inside out_file text; cut at space */
    char *sp = strchr(p->in_file, ' ');  if (sp) *sp = '\0';
    strncpy(p->command, trim(stage), sizeof(p->command) - 1);
}

void generate_execution_plan(const char *cmd_line) {
    printf("[Parsing Complex Command]: '%s'\n", cmd_line);
    printf("Execution Plan Tree:\n");

    char buf[256];
    strncpy(buf, cmd_line, sizeof(buf) - 1);
    buf[sizeof(buf) - 1] = '\0';

    char *stages[8];
    int n = 0;
    char *save;
    for (char *t = strtok_r(buf, "|", &save); t && n < 8; t = strtok_r(NULL, "|", &save))
        stages[n++] = t;

    for (int i = 0; i < n; i++) {
        StepPlan p;
        char original[128];
        snprintf(original, sizeof(original), "%s", stages[i]);   /* keep text for display */
```

```
        parse_stage(stages[i], &p);
        p.has_pipe_out = (i < n - 1);

        printf("  Stage %d: '%s'\n", i + 1, trim(original));
        if (p.in_file[0])  printf("           Input Source:  File ('%s')\n", p.in_file);
        else               printf("           Input Source:  Pipe %d [Read End]\n", i);
        if (p.out_file[0]) printf("           Output Target: File ('%s')\n", p.out_file);
        else if (p.has_pipe_out)
                           printf("           Output Target: Pipe %d [Write End]\n", i + 1);
        else               printf("           Output Target: Terminal (stdout)\n");
    }
    printf("Resolution Order: Resolved strictly left-to-right with descriptor chaining.\n");
}

int main() {
    printf("=== Week 10 Task 2: Complex Pipeline & Redirection Planner ===\n\n");
    const char *complex = "cat < input.txt | grep -i ossp | sort > final_report.txt";
    generate_execution_plan(complex);
    printf("\nExecution plan validated with zero syntax conflicts!\n");
    return 0;
}
```

Output:

```
charith@DESKTOP-RBQQSS:~$ nano s10.2.c
charith@DESKTOP-RBQQSS:~$ gcc s10.2.c -o s10.2
charith@DESKTOP-RBQQSS:~$ ./s10.2
=== Week 10 Task 2: Complex Pipeline & Redirection Planner ===

[Parsing Complex Command]: 'cat < input.txt | grep -i ossp | sort > final_report.txt'
Execution Plan Tree:
  Stage 1: 'cat < input.txt'
           Input Source:  File ('input.txt')
           Output Target: Pipe 1 [Write End]
  Stage 2: 'grep -i ossp'
           Input Source:  Pipe 1 [Read End]
           Output Target: Pipe 2 [Write End]
  Stage 3: 'sort > final_report.txt'
           Input Source:  Pipe 2 [Read End]
           Output Target: File ('final_report.txt')
Resolution Order: Resolved strictly left-to-right with descriptor chaining.

Execution plan validated with zero syntax conflicts!
charith@DESKTOP-RBQQSS:~$
```

TASK 3

Objective / Skill:

To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks.

Implementation / Work:

```
/*
 * Week 10 - Task 3: Background Job Detection & Non-Blocking Execution
 * Objective: To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately,
 *            Store Job Information, Monitor Execution, Test Background Tasks.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    printf("=== Week 10 Task 3: Background Job Launcher (&) ===\n\n");
    const char *cmd = "sleep 1 &";
    printf("Command entered: '%s'\n", cmd);

    size_t len = strlen(cmd);
    int background = (len > 0 && cmd[len - 1] == '&');
    if (background)
        printf("Background token '&' detected at end of line.\n");
    fflush(stdout);
```

```
    pid_t pid = fork();
    if (pid == 0) {
        // Background child process
        usleep(150000);   // 150ms work
        exit(0);
    }

    // Parent returns prompt immediately without waiting
    printf("[1] %d\n", pid);
    printf("myshell> <-- Prompt returned immediately without blocking!\n\n");

    // Monitor execution non-blockingly
    int status;
    pid_t result = waitpid(pid, &status, WNOHANG);
    if (result == 0) {
        printf("Non-blocking monitor: Background job %d is currently RUNNING.\n", pid);
    }

    // Wait for clean finish for test
    waitpid(pid, &status, 0);
    printf("Reaped background job %d upon completion.\n", pid);
    return 0;
}
```

Output:

```
charith@DESKTOP-RBQQSS:~$ nano s10.3.c
charith@DESKTOP-RBQQSS:~$ gcc s10.3.c -o s10.3
charith@DESKTOP-RBQQSS:~$ ./s10.3
=== Week 10 Task 3: Background Job Launcher (&) ===

Command entered: 'sleep 1 &'
Background token '&' detected at end of line.
[1] 694
myshell> <-- Prompt returned immediately without blocking!

Non-blocking monitor: Background job 694 is currently RUNNING.
Reaped background job 694 upon completion.
charith@DESKTOP-RBQQSS:~$
```

TASK 4

Objective / Skill:

To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records.

Implementation / Work:

```
/*
 * Week 10 - Task 4: Job Table Management & State Tracking
 * Objective: To Create Job Table, Store Process Groups, Track States,
 *            Update Status, Remove Completed Jobs, Validate Job Records.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef enum { JOB_RUNNING, JOB_STOPPED, JOB_DONE } State;

typedef struct {
    int jid;
    int pgid;
    char cmd[64];
    State state;
} JobEntry;

#define MAX_JOBS 8
JobEntry table[MAX_JOBS];

void print_table() {
    printf("  JID  PGID    State      Command\n");
    printf("  ------------------------------------------------\n");
    for (int i = 0; i < MAX_JOBS; i++) {
        if (table[i].jid != 0) {
            const char *s = (table[i].state == JOB_RUNNING) ? "Running" :
                            (table[i].state == JOB_STOPPED) ? "Stopped" : "Done";
```

```
            printf("  [%d]  %-6d  %-10s %s\n", table[i].jid, table[i].pgid, s, table[i].cmd);
        }
    }
}

int main() {
    printf("=== Week 10 Task 4: Job Table Management ===\n\n");
    memset(table, 0, sizeof(table));

    printf("[Action] Adding Job 1 and Job 2 to table:\n");
    table[0] = (JobEntry){1, 41021, "find / -name ossp &", JOB_RUNNING};
    table[1] = (JobEntry){2, 41025, "sleep 100", JOB_STOPPED};
    print_table();

    printf("\n[Action] Updating Job 1 status to Done and removing from table:\n");
    table[0].state = JOB_DONE;
    table[0].jid = 0;   // Removed
    print_table();

    printf("\nJob table state management validated!\n");
    return 0;
}
```

Output:

```
charith@DESKTOP-RBQQSS:~$ nano s10.4.c
charith@DESKTOP-RBQQSS:~$ gcc s10.4.c -o s10.4
charith@DESKTOP-RBQQSS:~$ ./s10.4
=== Week 10 Task 4: Job Table Management ===

[Action] Adding Job 1 and Job 2 to table:
  JID  PGID    State      Command
  ------------------------------------------------
  [1]  41021   Running    find / -name ossp &
  [2]  41025   Stopped    sleep 100

[Action] Updating Job 1 status to Done and removing from table:
  JID  PGID    State      Command
  ------------------------------------------------
  [2]  41025   Stopped    sleep 100

Job table state management validated!
charith@DESKTOP-RBQQSS:~$
```